API движка захвата

Английский оригинал: CAPTURE_API.md.

Движок захвата превращает файл PCAP или PCAPNG в нормализованный захват: набор TCP-сессий, каждая с двумя направленными потоками, где каждый наблюдаемый байт несёт свой провенанс. Это первая стадия конвейера; стадии протокола и интерфейса потребляют её выход и сами никогда не трогают файл захвата.

Модуль: `src.capture`. Все публичные имена реэкспортируются из `src.capture`.

Содержание

- Быстрый старт
- Типичные сценарии
- Публичный API
- Чтение провенанса
- Диагностики
- Ограничения
- Заметки о dpkt
- Производительность

Быстрый старт

Установите пакет с дополнениями для разработки:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

Сгенерируйте синтетические фикстуры и корпус (оба детерминированы):

```
python -m scripts.generate_fixture
python -m scripts.generate_corpus
```

Нормализуйте захват в контрактный JSON и проверьте его:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

То же из Python:

```
from pathlib import Path
from src.capture import export_capture, normalize

capture = normalize(Path("tests/corpus/corpus_capture_01.pcapng"))
export_capture(
    capture.sessions,
    Path("out/corpus_01.json"),
    source_file="captures/corpus_capture_01.pcapng",
    capture_id=capture.capture_id,
    streams=capture.streams,
)
```

Типичные сценарии

Разобрать захват. Вызовите `normalize(path)`, чтобы за один шаг получить сессии, оба направленных потока на сессию и диагностики. Используйте один `read_capture(path)`, когда нужны только пакеты, например чтобы их посчитать.

Обработать большой захват. Добавьте `--stream` в CLI (или вызовите `process_streaming`), что освобождает сессию из памяти, как только обе стороны закрыты FIN или RST. Сессия, которая никогда не закрывается, всё равно держится целиком, поэтому пик никогда не превышает конвейер в памяти. См. «Производительность» ниже.

Выгрузить в Wireshark. Добавьте `--wireshark-pcap out.pcap`, чтобы записать по одному пакету на направление сессии с собранными байтами, так что поток можно посмотреть в Wireshark рядом с исходным файлом. Это вид Follow TCP Stream без интерактивного шага.

Работа с дефектным захватом. `read_capture` читает повреждённый, но распознаваемый файл насколько может и сообщает остаток как диагностики вместо исключения. Пропуски и конфликтующие перекрытия в потоке появляются в `DirectionalStream.diagnostics`; проблемы парсера (усечение, фрагменты, смешанные типы канального уровня) появляются в списке диагностик, переданном в `read_capture`. Фикстуры в `tests/fixtures/defects/` покрывают каждый случай.

Публичный API

`read_capture(path, diagnostics=None, *, verify_checksums=False) -> Iterator[Packet]`

Читает `path` и выдаёт каждый TCP/IPv4 пакет в порядке файла.

- `diagnostics` — необязательный список; замечания о пропущенных типах канального уровня, кадрах IPv6 и не-TCP кадрах дописываются в него. Итератор не затрагивается.
- `verify_checksums` — когда false (по умолчанию), контрольные суммы не проверяются. Когда true, `Packet.checksum_valid` выставляется по каждому пакету; вызывающий решает, что делать с несовпадением.

Выбрасывает `FileNotFoundError`, если файл отсутствует, и `ValueError`, если файл не PCAP и не PCAPNG.

`Packet` (dataclass, frozen)

Поле	Тип	Смысл
`index`	`int`	позиция пакета в файле, с нуля
`timestamp`	`float`	время захвата, секунды эпохи
`src_ip`, `dst_ip`	`str`	адреса IPv4 как текст
`src_port`, `dst_port`	`int`	порты TCP
`seq`, `ack`	`int`	сырые 32-битные номера последовательности и подтверждения
`flags`	`int`	сырые флаги TCP
`payload`	`bytes`	нагрузка TCP, пустая для служебных сегментов
`is_truncated`	`bool`	захваченная нагрузка короче, чем обещал заголовок
`checksum_valid`	`bool | None`	выставляется только при проверке контрольных сумм

Удобные свойства: `syn`, `fin`, `rst`, `ack_flag`, `payload_len`.

`Diagnostic` (dataclass, frozen)

Поле	Тип	Смысл
`type`	`str`	один из типов в таблице раздела «Диагностики»
`packet_index`	`int | None`	пакет, к которому относится замечание, когда известен
`offset`	`int | None`	смещение потока, для замечаний уровня потока
`length`	`int | None`	длина в байтах, для замечаний уровня потока
`detail`	`str`	человекочитаемое объяснение

`to_contract()` возвращает словарь, записываемый в нормализованный JSON.

`build_sessions(packets) -> list[Session]`

Группирует пакеты в TCP-сессии. Сессия — это экземпляр соединения, а не 5-кортеж: если та же пара адрес/порт позже переиспользуется, это вторая сессия. Сессии появляются в порядке появления их первого пакета в файле, а `session_id` — `s1`, `s2` и так далее.

Роли — `client`/`server` только когда был виден SYN; иначе обе `unknown`.

`Session` (dataclass)

Поле	Тип	Смысл
`session_id`	`str`	`s1`, `s2`, …
`endpoints`	`tuple[Endpoint, Endpoint]`	две стороны, A и B
`syn_seen`, `fin_seen`, `rst_seen`	`bool`	флаги рукопожатия и завершения
`isn_a`, `isn_b`	`int | None`	начальные номера последовательности, когда был виден SYN
`first_ts`, `last_ts`	`float | None`	границы времени захвата
`packet_indices`	`list[int]`	индексы пакетов сессии, в порядке файла
`role_a`, `role_b`	`str`	`client`, `server` или `unknown`

Методы: `endpoint_index(ip, port)`, `direction_of(packet)`, `to_contract()`.

`Direction` (enum)

`A_TO_B` и `B_TO_A`. `Direction.A_TO_B.opposite` даёт другое.

`Endpoint` (dataclass, frozen)

`ip: str`, `port: int`. `to_contract()` возвращает `{"ip": ..., "port": ...}`.

`reassemble(session, direction, packets, *, ignore_checksums=True) -> DirectionalStream`

Пересобирает одно направление `session`. `packets` может содержать весь захват; пакеты, не принадлежащие сессии или направлению, игнорируются.

Алгоритм: сегменты упорядочиваются по номеру последовательности, а не по времени прибытия. Одинаковые ретрансмиссии добавляют провенанс, но не байты. Сегменты не по порядку буферизуются и склеиваются. Дыра становится диагностикой `gap`, никогда нулевыми байтами. Конфликтующее перекрытие сохраняет первые байты и добавляет диагностику `ambiguity`. Переход через границу 32-битного номера последовательности обрабатывается.

`DirectionalStream` (dataclass)

Поле	Тип	Смысл
`direction`	`Direction`	в какую сторону течёт этот поток
`bytes_`	`bytearray`	только наблюдаемые байты, дыры опущены
`provenance`	`Provenance`	отображение смещения на пакет
`diagnostics`	`list[Diagnostic]`	пропуски, неоднозначности, заметки о контрольных суммах
`packet_indices`	`list[int]`	пакеты, несшие это направление

Методы: `length`, `gaps()`, `ambiguities()`.

`Provenance` и `Range`

`Range` (dataclass, frozen): `offset`, `length`, `packet_index`, `seq`, `ts`. `end` — это `offset + length`. `to_contract()` возвращает словарь для JSON.

`Provenance`: `ranges: list[Range]`, плюс:

- `add(rng)` — дописать диапазон.
- `lookup(offset) -> Range | None` — диапазон, покрывающий смещение потока, или `None`, когда смещение попадает в дыру или за конец.
- `split(start, end) -> list[Range]` — диапазоны, обрезанные до `[start, end)`, с `offset` и `seq`, сдвинутыми так, что каждая часть всё ещё указывает на нужные байты.

`export_capture(sessions, out_path, *, source_file, capture_id, streams=None) -> None`

Пишет нормализованный захват как JSON, потоком, сессия за сессией. `streams` отображает id сессии на её два направленных потока.

`export_reassembled_pcap(sessions, out_path, *, streams=None, packets=None) -> int`

Пишет по одному пакету на направление сессии с собранными байтами, так что поток можно посмотреть в Wireshark рядом с исходным захватом. Возвращает число записанных пакетов. Поля заголовка, которых поток не несёт, фиксированы и документированы в модуле. CLI выставляет это как `--wireshark-pcap`.

Помощники

- `sha256_file(path) -> str` — `sha256:<hex>` файла, используется как `capture_id`.
- `capture_id_from_bytes(data) -> str` — то же из байтов, уже в памяти.
- `normalize(path) -> NormalizedCapture` — читать, построить сессии и пересобрать за один вызов. `normalize_bytes(data, source_file=...)` делает то же из байтов. `normalize_packets(...)` переиспользует уже прочитанные пакеты.

`NormalizedCapture`: `capture_id`, `source_file`, `sessions`, `streams` (`dict[session_id, dict[Direction, DirectionalStream]]`), `diagnostics` и `packets_read`.

Полный пример

```
from pathlib import Path

from src.capture import Direction, export_capture, normalize

capture = normalize(Path("tests/corpus/corpus_capture_01.pcapng"))

for session in capture.sessions:
    streams = capture.streams[session.session_id]
    a_to_b = streams[Direction.A_TO_B]
    print(session.session_id, len(a_to_b.bytes_), "bytes")

export_capture(
    capture.sessions,
    Path("out/corpus_01.json"),
    source_file="captures/corpus_capture_01.pcapng",
    capture_id=capture.capture_id,
    streams=capture.streams,
)
```

То же из командной строки:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

Чтение провенанса

По смещению потока `lookup` возвращает пакет, из которого он пришёл:

```
from src.capture import Direction, normalize

capture = normalize(Path("capture.pcapng"))
stream = capture.streams["s1"][Direction.A_TO_B]

rng = stream.provenance.lookup(4)
if rng is None:
    print("смещение 4 — дыра или за концом")
else:
    print(
        f"байт 4 пришёл из пакета {rng.packet_index} "
        f"на seq {rng.seq}, захвачен в {rng.ts}"
    )
```

Чтобы покрыть диапазон байт, пересекающий пакеты или дыры, используйте `split`; он возвращает один `Range` на каждый вносящий вклад кусок и никогда не выдумывает байты:

```
for piece in stream.provenance.split(start, end):
    print(piece.offset, piece.length, piece.packet_index, piece.seq)
```

Диагностики

type	Кем испускается	Смысл
`gap`	пересборка	ненаблюдённые байты; `offset` и `length` задают дыру
`ambiguity`	пересборка	одни и те же байты встретились с двумя разными значениями; первые сохранены, оба названы в `detail`
`checksum_offload`	пересборка	несовпадение контрольной суммы TCP, только при проверке сумм
`ipv6_ignored`	парсер	сохранён для файлов, записанных до разбора IPv6; больше не испускается
`non_ip`	парсер	кадр, не являющийся ни IPv4, ни IPv6
`non_tcp`	парсер	пакет IP, не являющийся TCP
`ip_fragment`	парсер	сохранён для файлов, записанных до сборки фрагментов; больше не испускается
`ip_fragment_reassembled`	парсер	фрагментированная датаграмма завершена; `detail` называет её пакеты-фрагменты
`ip_fragment_incomplete`	парсер	фрагментированная датаграмма так и не завершилась; `offset` и `length` задают дыру
`ip_fragment_overlap`	парсер	два фрагмента разошлись на одних байтах; IPv4 сохраняет первое прибытие, IPv6 отбрасывает датаграмму
`truncated_packet`	парсер	захваченная нагрузка короче, чем объявляет заголовок
`truncated_frame`	парсер	кадр был короче, чем утверждали его заголовки, или заголовок захвата повреждён
`unsupported_linktype`	парсер	захват использует тип канального уровня, отличный от Ethernet, loopback или cooked, либо смешивает типы

Диагностики парсера попадают в `NormalizedCapture.diagnostics`. Диагностики потока попадают в `DirectionalStream.diagnostics` и записываются в JSON вместе с потоком. Усечение также выставляется по каждому пакету как `Packet.is_truncated`.

Диагностика `gap` помечает точку, где поток продолжается после пропущенного участка: `offset` — позиция в `bytes_` байта, следующего за дырой, а `length` — сколько байтов пропущено перед ним. Значит, следующий байт тоже принадлежит потоку, а `offset - length` — место начала дыры в восстановленном порядке. Сам `bytes_` никогда не содержит байтов для дыры.

Чтение никогда не останавливается на дефекте. Захват, обрывающийся посреди блока, обрезанный заголовок секции, смешанные типы канального уровня, фрагментированные датаграммы и смешанные версии IP читаются насколько возможно и сообщаются; исключение `ValueError` выбрасывает только файл, вовсе не являющийся захватом. `tests/fixtures/defects/` держит по одной фикстуре на дефект, а `tests/capture/test_deformed_captures.py` покрывает каждый.

Сборка фрагментов

Фрагментированная датаграмма собирается до того, как достигнет слоя TCP, поэтому сессия, которую сеть раздробила, становится видимой. Фрагменты группируются по источнику, назначению, идентификации и протоколу (IPv4) или по источнику, назначению и идентификации

(IPv6); смещение фрагмента — в единицах по восемь байт. Датаграмма производится только когда покрыт каждый байт от нуля до объявленного конца и был виден последний фрагмент.

- Перекрытие с разными байтами сохраняет первое прибытие для IPv4 и сообщает `ip_fragment_overlap`; для IPv6 вся датаграмма отбрасывается, чего и требует RFC 5722, чтобы убрать неоднозначность.
- Датаграмма, которая так и не собралась, сообщается как `ip_fragment_incomplete` с `offset` и `length` дыры; её байты не домысливаются.
- Нефинальный фрагмент IPv4, чья длина не кратна восьми, испорчен и не собирается.
- Сборка для этой цели полная: фрагменты одной датаграммы могут быть разделены посторонним трафиком и доставлены не по порядку. Таймеры не применяются, потому что захват читается офлайн.

Ограничения

- IPv6 разбирается. TCP-сессии поверх IPv6 дают потоки точно как IPv4, а контрольные суммы TCP для IPv6 проверяются, когда задан `--verify-checksums`. Фрагментация IPv6 собирается.
- Фрагментация IP собирается для обеих версий. Датаграмма, которая так и не собралась в файле, сообщается как incomplete, а не истекает по тайм-ауту.
- Зашифрованный трафик не расшифровывается. TLS и всё остальное поверх TCP предстаёт непрозрачными байтами.
- Понимаются только захваты Ethernet, Linux loopback и BSD cooked; другие типы канального уровня сообщаются как `unsupported_linktype` и пропускаются.
- Весь захват держится в памяти. Измеренный предел на эталонном профиле нагрузки см. в `docs/BENCHMARK.md`.

Заметки о dpkt

Три детали стоили времени на поиск и их стоит помнить при работе над парсером:

- `dpkt.pcapng.Reader.datalink` — метод, тогда как `dpkt.pcap.Reader.datalink` — атрибут. Парсер вызывает его, когда он вызываемый.
- Константа флага push — `dpkt.tcp.TH_PUSH`, а не `TH_PSH`.
- `dpkt.utils.inet_aton` не существует; используйте `socket.inet_aton`.

Производительность

Измерено на эталонной нагрузке (профиль в `docs/BENCHMARK.md`: 95.66 МиБ входа, 250k пакетов, 1000 сессий, 122.44 МиБ выходного JSON) в контейнере с 4 vCPU:

Режим	Настенное время	Пик RSS
В памяти (`normalize`)	11.56 с	354 МиБ
Потоковый (`--stream`)	11.22 с	224 МиБ

Оба с большим запасом внутри лимитов задания: 5 минут и 4 ГиБ. Потоковый режим снижает пик, освобождая закрытые сессии; выход он не меняет. Перезапустите `python -m scripts.run_benchmark --pcap .benchmark/profile.pcapng`, чтобы воспроизвести.